

Padrões de Projeto (State)

QXD0068 - Reuso de Software

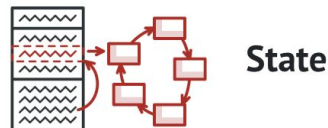
Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão State
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação

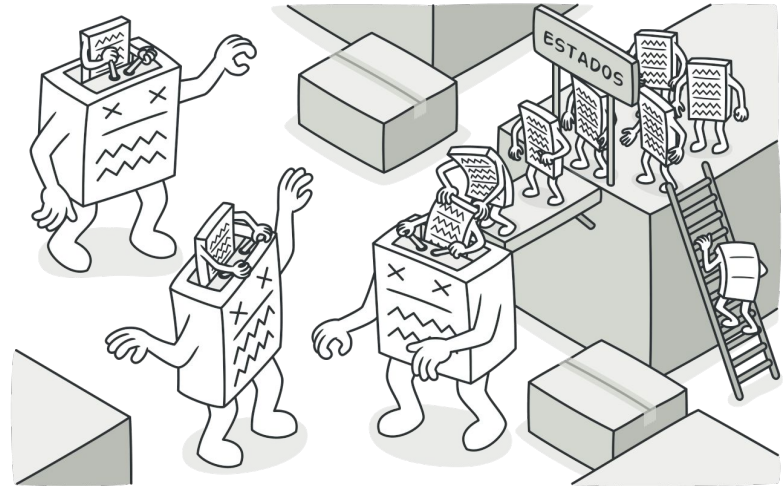


Introdução

- O State é um padrão de projeto comportamental que permite que um objeto altere dinamicamente seu comportamento quando o seu estado interno muda, como se o objeto mudasse de classe em tempo de execução.
- Esse padrão é especialmente útil quando uma classe possui muitos comportamentos condicionais dependentes de valores de estado, evitando grandes blocos de if ou switch.

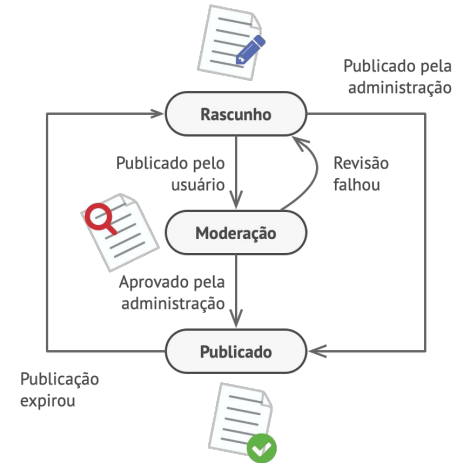
Padrão State

- O padrão State transforma o comportamento variável de um objeto em classes de estado independentes, todas implementando a mesma interface.
- O objeto principal (chamado Contexto) mantém uma referência para o estado atual e delegará a ele as operações.
- Assim, o comportamento do sistema muda automaticamente ao trocar o objeto de estado, sem condicionais e sem alterar o código do contexto.



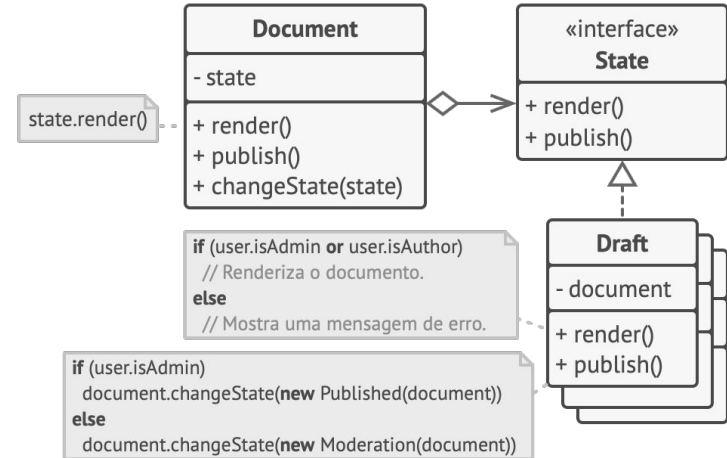
Problema

- Em muitos sistemas, um objeto pode assumir vários estados e se comportar de maneira diferente em cada um.
 - Por exemplo, um documento pode estar nos estados “Rascunho”, “Moderação” ou “Publicado”, e o método publicar() age de modo distinto em cada estado:
 - Em Rascunho, envia para moderação.
 - Em Moderação, publica se o usuário for administrador.
 - Em Publicado, não faz nada.
- Implementações tradicionais utilizam condicionais gigantes para tratar cada caso, tornando o código difícil de manter e estender.
- Com o tempo, essas condicionais se multiplicam, e cada alteração em um estado exige mudanças em várias partes do código, violando o Princípio Aberto/Fechado e tornando o sistema frágil.



Solução

- O padrão State propõe criar uma classe separada para cada estado. Cada uma delas implementa a interface State com os métodos que variam conforme o estado.
- O contexto (por exemplo, Documento ou AudioPlayer) mantém uma referência ao estado atual e delegará a ele as operações apropriadas.
- Quando ocorre uma transição de estado, o contexto simplesmente substitui o objeto de estado ativo por outro, sem precisar de condicionais. Cada classe de estado pode, inclusive, solicitar ao contexto que mude de estado, quando apropriado.
- Essa abordagem organiza o comportamento por estado, reduz a duplicação de código e facilita a inclusão de novos estados sem alterar os existentes.



Analogia do Mundo Real

- O comportamento dos botões de um smartphone muda conforme o estado do dispositivo:
 - Quando desbloqueado, os botões executam suas funções normais.
 - Quando bloqueado, pressionar um botão apenas liga a tela.
 - Quando a bateria está fraca, os botões mostram o alerta de energia.
- Os botões são os mesmos (contexto), mas o estado interno do telefone altera completamente suas respostas — exatamente o que o padrão State modela.

Estrutura do Padrão

- **Contexto:**
 - Armazena o estado atual.
 - Define métodos públicos que delegam o comportamento ao estado ativo.
 - Pode trocar dinamicamente seu estado chamando `changeState()`.

- **Interface State:**
 - Declara os métodos que cada estado deve implementar.
 - Define o contrato de comportamento entre o contexto e os estados.

- **Estados Concretos:**
 - Implementam o comportamento específico de cada estado.
 - Podem decidir quando realizar uma transição de estado no contexto.
 - Podem acessar o contexto, se necessário, para alterar o comportamento global.

- Essa estrutura elimina condicionais extensas e centraliza a lógica de cada estado em sua própria classe.

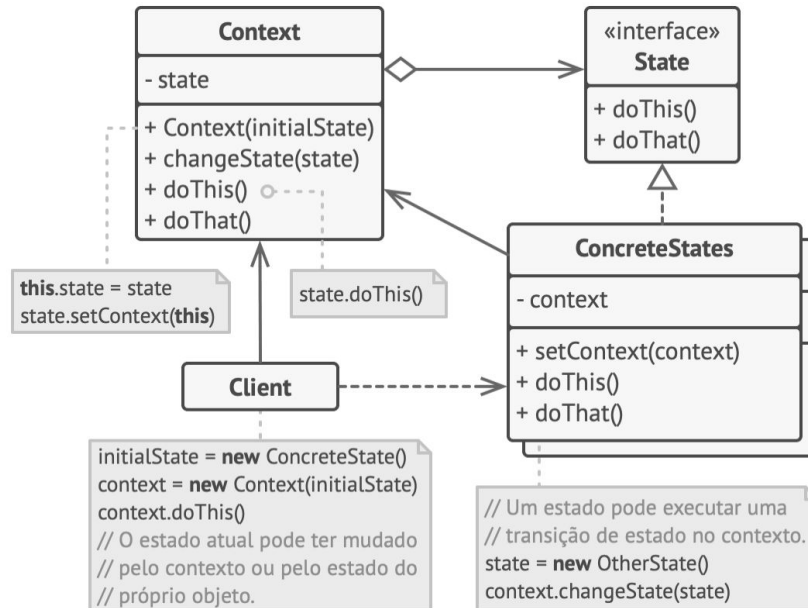
1 O **Contexto** armazena uma referência a um dos objetos concretos de estado e delega a eles todos os trabalhos específicos de estado. O contexto se comunica com o objeto estado através da interface do estado. O contexto expõe um setter para passar a ele um novo objeto de estado.

2 A interface do **Estado** declara métodos específicos a estados. Esses métodos devem fazer sentido para todos os estados concretos porque você não quer alguns dos seus estados tendo métodos inúteis que nunca irão ser chamados.

3 Os **Estados Concretos** fornecem suas próprias implementações para os métodos específicos de estados. Para evitar duplicação ou código parecido em múltiplos estados, você pode fornecer classes abstratas intermediárias que encapsulam alguns dos comportamentos comuns.

Objetos de estado podem armazenar referências retroativas para o objeto de contexto. Através dessa referência o estado pode buscar qualquer informação desejada do objeto contexto, assim como iniciar transições de estado.

4 Ambos os estados de contexto e concretos podem configurar o próximo estado do contexto e realizar a atual transição de estado ao substituir o objeto estado ligado ao contexto.



```
// Interface comum para todos os estados
interface State {
    fun play(player: AudioPlayer)
    fun pause(player: AudioPlayer)
}
```

```
// Contexto que mantém o estado atual
class AudioPlayer {
    var state: State = ReadyState()

    fun changeState(newState: State) {
        state = newState
    }

    fun play() = state.play(this)
    fun pause() = state.pause(this)
}
```

```
// Estados concretos
class ReadyState : State {
    override fun play(player: AudioPlayer) {
        println("Iniciando reprodução...")
        player.changeState(PlayingState())
    }

    override fun pause(player: AudioPlayer) {
        println("Nada está tocando para pausar.")
    }
}
```

```
class PlayingState : State {
    override fun play(player: AudioPlayer) {
        println("Já está tocando.")
    }

    override fun pause(player: AudioPlayer) {
        println("Pausando reprodução...")
        player.changeState(ReadyState())
    }
}

// Uso
fun main() {
    val player = AudioPlayer()
    player.play() // Muda para PlayingState
    player.pause() // Volta para ReadyState
}
```

Exemplo em Kotlin

```
from abc import ABC, abstractmethod

# Interface de Estado
class State(ABC):
    @abstractmethod
    def handle(self, context):
        pass

# Contexto
class Document:
    def __init__(self):
        self.state = DraftState()

    def set_s (variable) state: Any
        self.state = state

    def publish(self):
        self.state.handle(self)

# Estados concretos
class DraftState(State):
    def handle(self, context):
        print("Enviando documento para moderação...")
        context.set_state(ModerationState())

class ModerationState(State):
    def handle(self, context):
        print("Publicando documento aprovado...")
        context.set_state(PublishedState())

class PublishedState(State):
    def handle(self, context):
        print("Documento já está publicado.")
```

Exemplo em Python

Aplicabilidade

- **Use o padrão State quando:**
 - Um objeto tem múltiplos estados e comportamentos diferentes em cada um.
 - Há condicionais extensas controlando comportamentos.
 - Deseja-se adicionar novos estados facilmente, sem alterar o código existente.
 - Há repetição de código entre partes que poderiam ser isoladas em estados distintos.

- O padrão reduz a complexidade e aumenta a coesão, pois cada estado se torna uma unidade lógica independente.

Prós e Contras

- **Vantagens**

- Organiza o código relacionado a cada estado em classes próprias.
- Aplica os princípios de responsabilidade única e aberto/fechado.
- Elimina condicionais e facilita a adição de novos estados.
- Permite transições dinâmicas de estado em tempo de execução.

- **Desvantagens**

- Aumenta o número de classes no sistema.
- Pode ser exagerado se houver poucos estados ou mudanças raras.
- Requer planejamento cuidadoso para manter a clareza das transições.



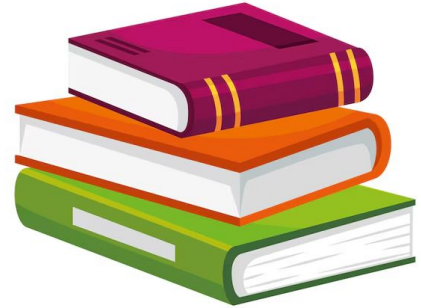
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado! Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

